If you're running models locally, thinking "model → VRAM" falls apart once you account for how the weights were trained and quantized in the first place.
There's a better way to think about it:
VRAM (in GB) ≈ Parameters (in billions) x (effective bits per weight ÷ 8)
That's it.
This one formula explains everything across:

- FP16 / BF16
- FP8 / INT8
- GPTQ / AWQ / NF4
- GGUF variants
- basically every format you'll use

The Only Conversion You Actually Need

Here's the core intuition:

- FP16 / BF16 → 16 bits → ~2 GB per 1B params
- FP8 / INT8 → 8 bits → ~1 GB per 1B params
- 4-bit quants → ~4 bits → ~0.5 GB per 1B params

GGUF formats sit in between depending on the exact scheme:

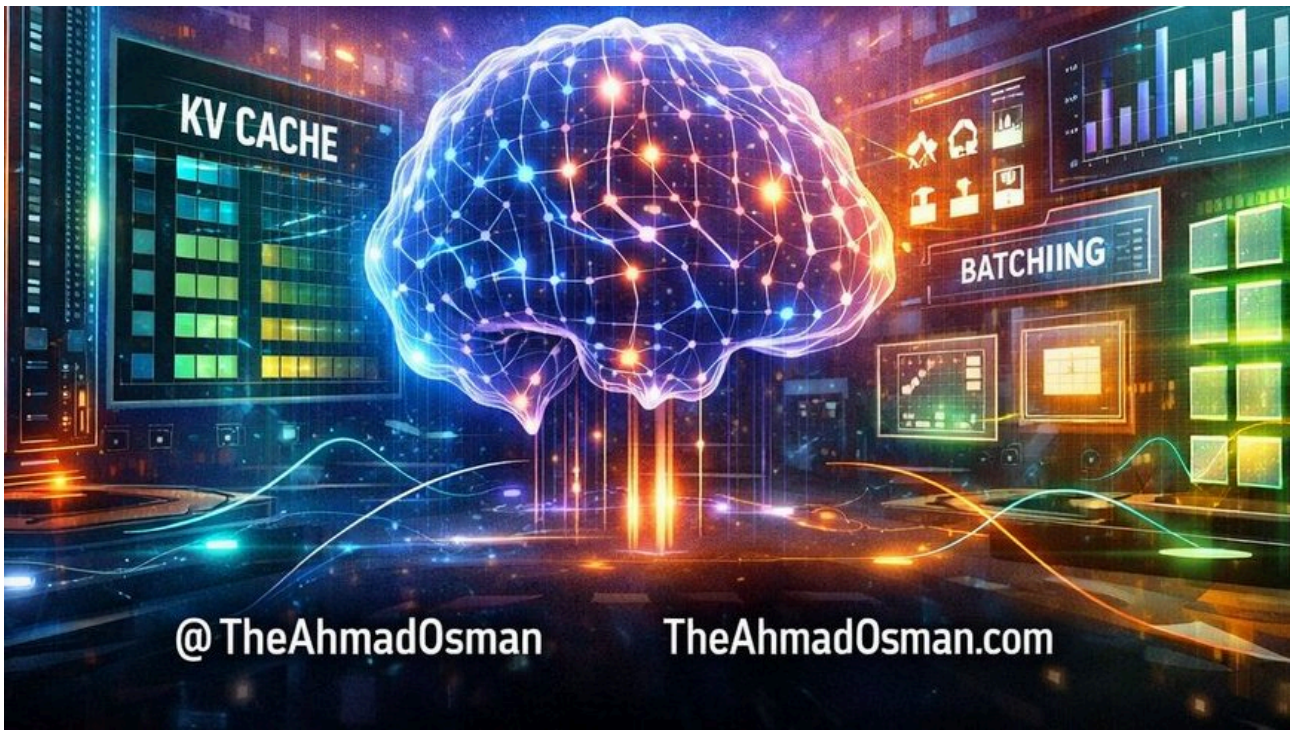
- Q6_K → ~0.82 GB per 1B
- Q5_K → ~0.69 GB per 1B
- Q4_K → ~0.56 GB per 1B
- Q3_K → ~0.43 GB per 1B
- Q2_K → ~0.33 GB per 1B

Ultra-aggressive quants go even lower, but at a cost.
If you remember nothing else, remember this:

- FP16 = 2x model size
- FP8 = 1x model size
- 4-bit = 0.5x model size

Everything else is just variations on that theme.

Side Note: The VRAM Tax Nobody Talks About



Before you even think about weights, understand this: the model itself is only part of your VRAM bill. The real killer is everything around it. KV cache grows with context length and will quietly eat your memory alive at 32K, 128K, or higher. Activations vary by runtime and optimization level but can spike under certain execution paths. Batching and concurrency multiply memory usage fast, especially in agent-style workloads. Framework overhead adds its own tax depending on whether you're using Transformers, vLLM, TensorRT-LLM, or llama.cpp. And then there's CUDA Graphs, which trade extra reserved memory for much better latency and throughput stability. Bottom line: if you only budget for weights, you're already out of memory.

What This Looks Like in Practice

Let's translate that into real model sizes.

A 7B model:

- FP16 → ~14 GB
- FP8 → ~7 GB
- 4-bit → ~3.5–4 GB

A 13B model:

- FP16 → ~26 GB
- FP8 → ~13 GB
- 4-bit → ~6–7 GB

A 70B model:

- FP16 → ~140 GB
- FP8 → ~70 GB
- 4-bit → ~35–40 GB

A 405B model:

- FP16 → ~810 GB
- FP8 → ~405 GB
- 4-bit → ~200+ GB

Now you understand why people either:

- quantize aggressively
- shard across GPUs (e.g. Tensor Parallelism)
- or just give up and say "cloud it is"

GPU Reality: What Actually Fits

Here's the practical translation into GPUs people actually own.

8 GB VRAM:

- ~3B in FP16
- ~6–7B in FP8
- ~12–13B in 4-bit

12 GB VRAM:

- ~5B FP16
- ~10B FP8
- ~18–20B 4-bit

16 GB VRAM:

- ~7B FP16
- ~13B FP8
- ~25B 4-bit

24 GB VRAM:

- ~10–12B FP16
- ~20B FP8
- ~35–40B 4-bit

48 GB VRAM:

- ~20–24B FP16
- ~40B FP8
- ~70–80B 4-bit

80 GB VRAM:

- ~35–40B FP16
- ~70B FP8
- ~140B-class 4-bit

This is the "what actually fits" version for model weights.

Why Your Model Still Crashes

As we said earlier, even if the math says it fits, you can still run out of memory.

Because weights are only part of the story.

You also need memory for:

- KV cache (this explodes with long context)
- activations (depending on runtime)
- batching / concurrency
- framework overhead

Rule of thumb:

Add 10–30% extra VRAM for a safe run.

If you're doing:

- long context (32K, 128K, etc)
- high concurrency
- agent workflows

…you'll need even more.

The MoE Trap

Mixture-of-Experts models confuse people.

Example:

- "8x7B" sounds like 56B
- but only a subset of experts run per token

So compute cost ≠ memory cost.

What matters:

- total parameters → affects memory footprint

- active parameters → affects speed

Depending on how the model is loaded:

- you may still need memory for all experts
- or you can shard them across GPUs

If you treat MoE like dense, you'll either overestimate or underestimate badly.

GGUF Is Not Magic

GGUF gets treated like a cheat code.
It's not.
It's a container + quantization strategy optimized for:

- llama.cpp-style inference
- CPU + GPU hybrid setups
- ultra-efficient memory usage

But:
Those memory numbers only apply in that runtime.
The moment you move into other frameworks:

- weights may be dequantized
- memory usage can jump dramatically

So "it fits in 6 GB" is not universal truth. It's runtime-specific truth.

The Only Mental Model That Matters

There isn't a giant compatibility matrix you need to memorize.
There's just this:
VRAM ≈ B x (bits ÷ 8)
Then adjust for:

- runtime overhead
- KV cache
- concurrency

That's it.
Once you internalize this, you stop guessing.
You start designing systems.
And more importantly, you stop asking: "Can I run this?"
You start asking: "How do I want to run this?"
That's when things get interesting.